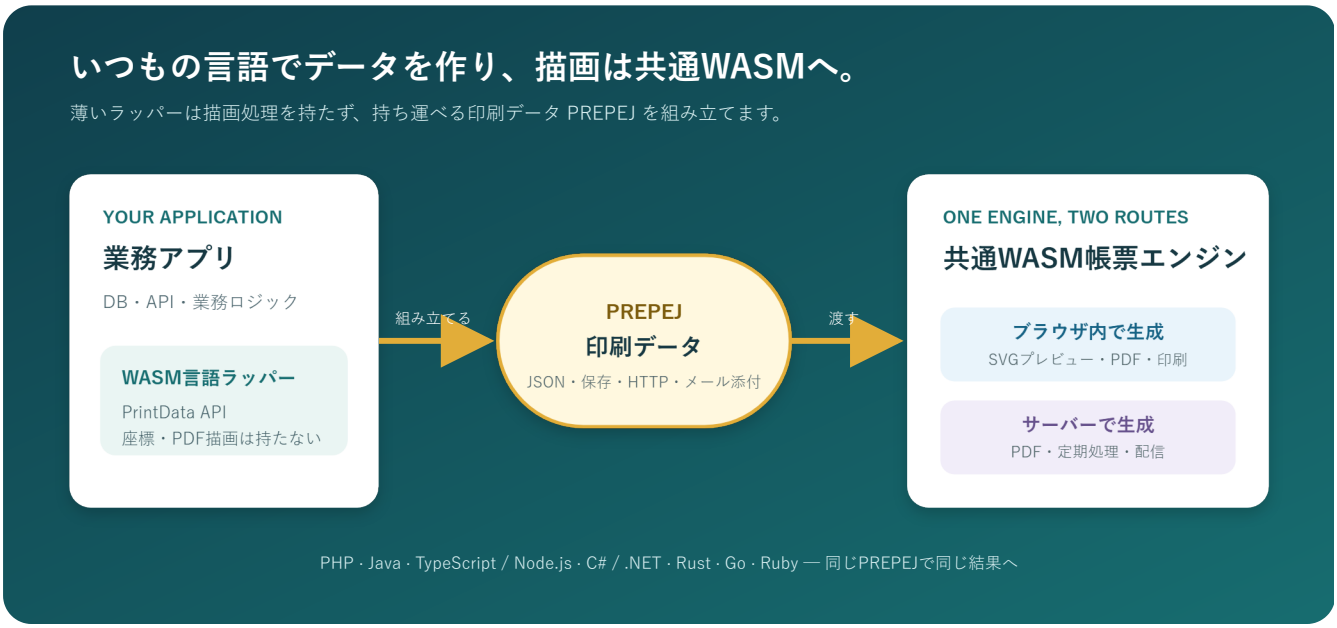
# Reports.web Go WASM言語ラッパー プログラマーズガイド／APIリファレンス

Reports.webのWASM言語ラッパーは、Goの業務データから、共通の印刷データPREPEJを組み立てるための小さなライブラリです。帳票の座標計算やPDF描画を各言語へ重複実装せず、ブラウザまたはサーバーの共通WASM帳票エンジンへ任せます。

この一冊では、初めて利用する方が次のプログラムを作れるようになることを目標にしています。

- Goから帳票定義PREPDJへ業務データを流し込む
- 完成した印刷データPREPEJをJSON、ファイル、HTTPで受け渡す
- ブラウザでSVGプレビュー・印刷・PDF生成を行う
- サーバー側の共通WASMエンジンでPDFを生成・配信する
- Goラッパーの主要APIを調べる



# 目次


1. 最初に知っておくこと 3

    1.1 「WASM言語ラッパー」とは 3

    1.2 帳票ができるまで 3

    1.3 二つのファイル 3

2. サンプルを動かす 3

3. 最初の一枚を作る 4

4. PrintData APIリファレンス 5

    4.1 値とインデックス 5

    4.2 ページごとに定義を切り替える 6

5. 明細と改ページ 7

    5.1 空文字で罫線・図形を消す 7

    5.2 何も書かない繰返しは1行目だけ描く 8

6. 画像・バーコード・動的属性 8

7. ブラウザへ渡して表示する 8

8. PDFを作る二つの場所 9

    8.1 ブラウザで作る 9

    8.2 サーバーで作る 9

9. PREPEJを保存・再利用する 10

10. Web APIとデータベース 10

11. テストとリリース確認 11

12. よくある問題 11

13. 配置・更新・ライセンス 12

# 1. 最初に知っておくこと

## 1.1 「WASM言語ラッパー」とは

本書では、各言語でPREPEJを組み立てる補助ライブラリを**WASM言語ラッパー**と呼びます。Go版の実装は`printdata`パッケージの`Builder`です。ラッパー自身はWASMを直接ロードせず、SVGやPDFも描きません。出力形式を共通JSONに揃えることで、業務処理と描画エンジンを分けます。

これはPure Java／Pure Python／WPF・.NETの言語別エンジンとは異なる方式です。Pure系は各言語の帳票エンジンを直接呼びます。WASM方式は7言語が同じWASMエンジンとブラウザUIを共有します。

## 1.2 帳票ができるまで

1. デザイナーで帳票定義 `.prepdj` を作ります。
2. Goの`printdata.Builder`へPREPDJを設定します。
3. DBやAPIから取得した値を、デザイン上のオブジェクト名へ設定します。
4. 1ページずつ確定し、`.prepej`を作ります。
5. PREPEJをブラウザのプレビュアー、またはサーバーWASMへ渡します。
6. 共通エンジンがSVGプレビュー、PDF、印刷を担当します。

## 1.3 二つのファイル

| 拡張子 | 呼び名 | 役割 |
|---|---|---|
| `.prepdj` | 帳票定義 | 用紙、文字、罫線、画像、バーコード、繰返しなどのデザイン |
| `.prepej` | 印刷データ（Print Document） | 帳票定義とページごとの値・属性をまとめた、再表示・再印刷可能な文書 |

PREPEJは単なる一時レスポンスではありません。ファイル保存、再読込、メール添付、REST API、サーバー間転送、後日の再印刷に同じ文書を使えます。

# 2. サンプルを動かす

基準環境は **Go 1.25** です。同梱のDocker構成では必要な言語環境とDBをコンテナ内に用意するため、まずPC側で準備するのはDockerです。Windows／MacではDocker Desktopを起動し、LinuxではDocker EngineとComposeプラグインが使える状態にしてください。

ZIPを全体ごと展開してください。共通RuntimeとDB初期化データも使うため、番号フォルダーだけを取り出さないでください。初回のイメージ取得とビルドにはインターネット接続が必要です。

短いコードから学ぶ場合は `WebAssembly/Samples/go/01.`あっという間に帳票出力`/README.html` から始めます。ここでは全帳票を確認できる02の起動を説明します。

1. VS CodeやCursor、または使い慣れたエディターで `WebAssembly/Samples/go/02.`全帳票を`Web`でプレビュー を開きます。
2. そのフォルダーでターミナルを開き、次を実行します。開発用のReports Webリポジトリをビルドする必要はありません。

```
docker compose up -d --build --wait
docker compose ps
```

3. 起動が成功してから、ブラウザで `http://127.0.0.1:19251/` を開きます。URLを開くだけではアプリは起動しません。
4. 終了するときは、同じフォルダーで `docker compose down` を実行します。このサンプルのコンテナとネットワークが削除されます。DBデータの初期化やイメージの片付けは、そのフォルダーのREADMEに手順があります。

02では、単純帳票、10の倍数、郵便番号一覧、見積書、請求書、商品一覧、名刺、デザイン機能見本を確認できます。請求書などを選び、ソースと出力結果を見比べてください。別のアプリから帳票APIを呼ぶ使い方は、続く03のREADMEに説明しています。

## 3. 最初の一枚を作る

01の`main.go`にある`printData()`関数を、次の関数全体で置き換えます。既存の`import`と`main()`は残してください。必要な`printdata/`、`go.mod`、帳票定義は01に同梱されています。

```
func printData() ([]byte, error) {
    raw, err := os.ReadFile("definition/quick-report.prepdj")
    if err != nil { return nil, err }
    var definition printdata.Map
    if err = json.Unmarshal(raw, &definition); err != nil { return nil, err }

    report := printdata.New()
    if err = report.SetDefinition(definition); err != nil { return nil, err }
    if err = report.PageStart(nil); err != nil { return nil, err }
    if err = report.SetValue("Title", "はじめての帳票"); err != nil { return nil, err }
    if err = report.SetValue("CustomerName", "パオ商事株式会社"); err != nil { return nil, err }
    if err = report.PageEnd(); err != nil { return nil, err }
    return report.JSON(true)
}
```

01のフォルダーで`docker compose up -d --build --wait --wait-timeout 180`を実行し、成功後に`http://127.0.0.1:19250/`を再読み込みします。「はじめての帳票」と会社名が表示され、「PDFを保存」で同じ内容を保存できます。`Title`と`CustomerName`は同梱定義内の名前です。自作の帳票ではデザイナー上のオブジェクト名に合わせます。

呼出順は **定義を設定 → ページを開始 → 値を設定 → ページを確定 → JSONを取得** です。未完了のページがある状態ではPREPEJを取得できません。この制約により、途中までの帳票を誤って配信することを防ぎます。

## 4. PrintData APIリファレンス

配布ZIP内の実装ファイルは `WebAssembly/Samples/go/02.全帳票をWebでプレビュー/printdata/printdata.go` です。各番号のサンプルにも必要なソースを置いています。次のAPI名は実装に合わせています。

| API | 用途 |
| --- | --- |
| `SetDefinition` | 帳票定義（PREPDJ）を設定します。設定値は複製され、呼び出し元の変更から守られます。 |
| `PageStart` | 新しい印刷ページを開始します。別の定義を渡すと、表紙から明細などへ様式を切り替えられます。 |
| `SetValue / SetValueAt` | 名前で指定した動的オブジェクトへ値を設定します。繰返し位置を指定する場合はSetValueAtを使います。 |
| `SetRepeatedValue` | 縦横二方向に繰り返すオブジェクトへ、X・Y位置を指定して値を設定します。 |
| `SetTypedValue` | DynamicText、DynamicImage、DynamicBarcodeなど、値の種類を明示して設定します。 |
| `SetImage` | 画像URLまたはdata URIを動的画像へ設定します。 |
| `SetBarcode` | バーコード／二次元コードへ渡す文字列を設定します。 |
| `ChangeAttributes` | 位置、幅、色、フォント、配置などをページ単位で動的に変更します。 |
| `ChangeRepeatedAttributes` | 繰返し位置を含めて動的属性を変更します。 |
| `PageEnd` | 現在のページを確定し、印刷データへ追加します。 |
| `Value / JSON` | 完成したPREPEJをオブジェクトまたはJSON文字列として取得します。 |
| `Save` | 完成したPREPEJを一時ファイル経由で安全に保存します。 |

### 4.1 値とインデックス

`printdata.New()`は`*Builder`を返します。次の変更メソッドは成功時に`nil`、失敗時に`error`を返すため、エラーを確認してから次へ進みます。HTTP要求ごとに新しいBuilderを作り、複数の利用者で使い回さないでください。

| 呼び出し | 引数と戻り値 |
| --- | --- |
| `SetDefinition(d Map)` | JSONを読み込んだ定義を設定 |

| 呼び出し | 引数と戻り値 |
| --- | --- |
| `PageStart(d Map)` | `nil`なら現在の定義、新しい定義ならそのページから切替 |
| `SetValue(name string, value any)` | 通常の文字・数値。位置0、描画あり |
| `SetValueAt(name string, value any, index int, drawing bool)` | 1方向の繰返し位置と描画有無 |
| `SetRepeatedValue(name string, value any, x, y int, drawing bool)` | 2方向の繰返し位置 |
| `SetTypedValue(name, kind string, value any, index int, drawing bool)` | 種類も指定。例：`DynamicImage`、`DynamicBarcode` |
| `SetImage(name string, value any)` / `SetBarcode(name string, value any)` | 画像／バーコードを位置0へ設定 |
| `ChangeAttributes(name string, values Map, index int)` | 動的文字の属性を設定 |
| `ChangeRepeatedAttributes(name string, values Map, index int, ix, iy *int, kind string)` | 種類と繰返し位置も指定。2方向を使わないときはix/iyを両方nilにする |
| `PageEnd()` | ページを確定 |
| `Value()` | `(Map, error)`。完成した印刷データを取得 |
| `JSON(pretty bool)` | `([]byte, error)`。trueなら読みやすい改行付きJSON |
| `Save(path string)` | `error`。サーバー側のファイルへ保存。保存先ディレクトリは事前に用意 |

`Map`は`map[string]any`の別名です。`Save`は利用者のブラウザーにダウンロードする機能ではありません。ダウンロードは01の画面や03のHTTP応答を参考にします。

値には文字列、数値、真偽値、nullの単純値を渡します。配列や任意オブジェクトを値として渡さず、帳票オブジェクトごとに明示的に対応付けてください。繰返しのindex、indexX、indexYは0から始まり、負数はエラーです。

`drawing=false`は、その繰返し位置を描画しない場合に使います。値の設定はページのValuesへ追加されるため、同じ名前・同じ位置の値を何度も設定せず、業務処理で決めた最終値を一度渡します。属性の値にはnilを使えません。

## 4.2 ページごとに定義を切り替える

表紙と本文、途中から異なる一覧形式などでは、ページ開始時に別のPREPDJを渡します。座標をプログラムへ直接書くのではなく、デザイナーで完成させた定義をページへ結び付けます。定義を変更する前に現在のページを必ず確定してください。

## 5. 明細と改ページ

以下はerrorを返す帳票作成関数内の例です。definitionには「ページ」「品番」「品名」「金額」の動的文字を持つ定義、detailsにはCode・Name・Amountを持つ明細のスライスを用意します。第3章の1ページ定義へそのまま貼る例ではありません。実際の同梱帳票との対応は02のcatalog.goにあります。

```
print := printdata.New()
if err := print.SetDefinition(definition); err != nil { return err }
for offset, page := 0, 1; offset < len(details); offset, page = offset+20, page+1 {
    if err := print.PageStart(nil); err != nil { return err }
    if err := print.SetValue("ページ", page); err != nil { return err }
    end := min(offset+20, len(details))
    for row, d := range details[offset:end] {
        if err := print.SetValueAt("品番", d.Code, row, true); err != nil { return err }
        if err := print.SetValueAt("品名", d.Name, row, true); err != nil { return err }
        if err := print.SetValueAt("金額", d.Amount, row, true); err != nil { return err }
    }
    if err := print.PageEnd(); err != nil { return err }
}
```

1ページの行数は帳票定義と業務仕様から決めます。最終ページの空行を描画するか、明細ごとに罫線を出すかもプログラム側で明示します。ページ番号はPREPEJのページ順と、帳票へ表示する業務上のページ番号を分けて考えます。

### 5.1 空文字で罫線・図形を消す

罫線・四角・円（Line、DynamicLine、Square／Rectangle、Circle／Ellipse）は、値で表示を切り替えられます。

| 書く値 | 罫線・図形 |
| --- | --- |
| ""（空文字） | 描かない（消える） |
| nil、または何も書かない | 定義どおり描く |
| ""以外の値（例 "1"） | 描く。値は表示には使わない |

繰返しの行でも、1つだけの図形でも同じです。最初に書いたときも、別の値のあとに書いたときも同じです。表の行を途中で終える、合計行の上だけ線を引かない、条件で枠を消す、などに使います。次の明細線は、定義にある罫線のオブジェクト名の例です。

```
if err := print.SetValueAt("明細線", "", row, true); err != nil { return err } // この行の罫線を消す
```

drawing=falseも描きません。PREPEJでは、図形の値が""ならDrawingがtrueでも描きません。

文字の欄（Text）は別の規則です。値が""または`nil`なら、文字も枠線も背景色も描きません。SVG・印刷・PDFで同じです。

### 5.2 何も書かない繰返しは1行目だけ描く

繰返し（`RepeatCount`が2以上）のある物体に、そのページで何も書かないと、定義どおり1つだけ、1行目の位置に描きます。行の数だけ並べては描きません。

一部の行にだけ書くと、書いた行だけ描きます。書かなかった行は描きません。1行目に書かなければ、1行目も描きません。

表の罫線や枠を全部の行に描くには、描きたい行それぞれに`nil`（値なし）を書きます。罫線・図形は`nil`なら描き、""なら消えます（5.1）。次の1行を上の明細のループに足すと、明細の行数ぶん明細線に`nil`を書きます。

```
if err := print.SetValueAt("明細線", nil, row, true); err != nil { return err } // この行の罫線を描く
```

文字の欄は、`nil`を書くとその行の文字が空になります。定義の文字を出したい行には、その文字を書きます。PREPEJへ保存した印刷データでも同じです。

## 6. 画像・バーコード・動的属性

ページ開始後の操作例です。対応する動的画像「角印」、動的バーコード「商品QR」、動的文字「合計金額」を定義に用意します。`stampDataURI`は画像データの文字列です。ここも`error`を返す関数内の抜粋です。

```
if err := print.SetImage("角印", stampDataURI); err != nil { return err }
if err := print.SetBarcode("商品QR", "https://example.jp/items/123"); err != nil { return err }
if err := print.ChangeAttributes("合計金額", map[string]any{
    "bold": true, "fontSize": 16,
    "horizontalAlignment": "Right",
}, 0); err != nil { return err }
```

画像は閲覧先でも取得できる管理下のURL、または許容サイズのdata URIにします。サーバーPDF生成では、任意の外部URLをそのまま取得するとSSRFにつながります。固定画像は許可リストまたはアプリ管理下のファイルから解決してください。

動的属性では、名前・型・値を無検証のリクエストから直接渡さないでください。変更できる属性をアプリ側で限定し、数値範囲、色、配置、フォント名を検証します。

## 7. ブラウザへ渡して表示する

Go APIはPREPEJを`application/json; charset=utf-8`で返します。次は01の実在するURLを使った、表示部分の抜粋です。エラー表示やPDFボタンを含む完成した画面は01の`index.html`を参照してください。

```
<iframe id="reportsViewer" title="帳票プレビュー" style="width:100%;height:75vh;border:0"></iframe>
<script>
const viewer = document.querySelector('#reportsViewer');
viewer.addEventListener('load', async () => {
  const response = await fetch('/print-data');
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  await viewer.contentWindow.ReportsWeb.setPrintData(await response.json());
});
viewer.src = '/reports.web/preview/index.html?empty=1';
</script>
```

親画面とプレビュアーは同一オリジンに置くのが最も簡単です。別オリジンで`postMessage`を使う場合は、送信元と送信先のoriginを固定し、*を本番環境で使わないでください。

プレビュアー側では`setZoom("auto")`、ページ移動、検索、印刷、`exportPdf()`、`savePrintData()`を利用できます。独自画面へ表示面だけを組み込む場合は`hideAll()`でツール群を隠し、同じ機能を外側のボタンから呼び出します。

## 8. PDFを作る二つの場所

### 8.1 ブラウザで作る

PREPEJを利用者のブラウザへ渡し、ブラウザ内のWASMがPDFを生成します。サーバーのPDF描画負荷を抑え、プレビューからそのまま保存・印刷する操作に向きます。機密データをブラウザへ渡してよいかは、業務の認可設計で判断します。

### 8.2 サーバーで作る

net/httpなどから共通Node/WASMサービスへHTTPでPREPEJを送ります。 定期処理、メール添付、保存、監査、画面を開かない帳票生成に向きます。

```
POST /render/pdf
Content-Type: application/json

<PREPEJ JSON>
```

成功時は`application/pdf`を返します。失敗時は次の状態コードと、理由を書いたJSON（`{"error":"…"}`）を返します。

| 状態 | 意味 | 対処 |
| --- | --- | --- |
| 400 | 入力の誤り。JSONが壊れている、数値が有限の数でない、幅・高さなどが負、定義の数値を文字列で書いた、ルートの`Version`が無い・`"1.0"`のような文字列でない、など | `error`の理由を見て、PREPEJを直します |
| 413 | 大きすぎる要求。64MiBを超える、または64MiB以内でもサーバーのメモリーで処理できる量を超える | ページを分けて送るか、Node.jsのメモリー上限（`--max-old-space-size`）を増やします |
| 408 | 要求の本文が30秒以内に届かなかった。届かない本文で、後ろに並んだ要求を止めないためです | 送信側の接続を確かめて再実行します。時間は環境変数`REPORTS_ENGINE_BODY_TIMEOUT_MS`（ミリ秒）で変えられます |
| 503 | 混雑。要求は本文の受け取りから PDF の生成まで1件ずつ順番に処理し、処理の開始まで10秒を超えて待った要求を断ります。処理が5分を超えた要求も断り、次の要求へ進みます | しばらく待って再実行します。待ち時間は環境変数`REPORTS_ENGINE_QUEUE_WAIT_MS`、処理の時間の上限は`REPORTS_ENGINE_OPERATION_TIMEOUT_MS`（どちらもミリ秒）で変えられます |

408・413・503のときもサーバーは止まらず、次の要求を処理します。業務アプリ側でも処理時間を制限し、失敗時の途中PDFを保存・配信しないでください。ブラウザ生成とサーバー生成は同じPREPEJを入力にするため、業務データ組立てを二重実装する必要はありません。

## 9. PREPEJを保存・再利用する

`Save`は完成したPREPEJをUTF-8 JSONとして保存します。保存前にページをすべて確定してください。ファイル名には`.prepej`を使い、アップロード時には拡張子だけで信用せず、Format、Version、Definition、Pages、サイズ、ページ数を検証します。

保存した文書は、Webプレビュアーの「開く」またはドラッグ＆ドロップで再表示できます。HTTPで受け渡す場合もファイルと同じJSONです。再印刷の証跡として保管する場合は、業務データの保持期間、暗号化、アクセス権、削除規則を定めます。

## 10. Web APIとデータベース

DB列と帳票オブジェクト名はコードで明示的に対応付けます。SQLはプレースホルダーを使い、利用者が参照できる伝票かを取得前に確認します。日付、通貨、端数処理は帳票へ渡す前に業務規則で確定します。

機密帳票のレスポンスには`Cache-Control: no-store`を設定します。内部例外、接続文字列、ファイルパスを利用者へ返さず、要求IDを安全なサーバーログへ残します。帳票定義や画像は可能なら公開ルート外へ置きます。

## 11. テストとリリース確認

PCにGoを用意している場合は、02のフォルダーで`go test ./...`を実行できます。Dockerだけを用意した方は、各READMEの表示・文字変更・保存・再読込の手順で確認してください。

最低限、次を確認します。

- 共通印刷データの形式識別子を持ち、Versionが`1.0`である
- PREPDJを設定せずにページを開始するとエラーになる
- ページを閉じずに次のページを開始できない
- 日本語、改行、数値、真偽値、nullが壊れない
- 明細の先頭・最終行、改ページ前後に欠落や重複がない
- ページ途中の帳票定義切替が、完了済みページを変更しない
- 画像・角印、バーコード、動的属性がSVGとPDFへ反映される
- 保存したPREPEJを開き直すと同じページ・内容になる
- ブラウザ生成とサーバー生成のページ数・主要表示が一致する

ラッパー変更時はWASM描画エンジン全体を毎回再試験するのではなく、生成PREPEJが共通仕様と基準サンプルに一致することを中心に確認します。WASM、プレビュアー、フォント、画像処理を変更した場合は、別途描画回帰試験が必要です。

## 12. よくある問題

| 症状 | 確認すること |
| --- | --- |
| 帳票が空になる | PREPDJ上のオブジェクト名と`SetValue`の名前、indexを確認します。 |
| 2ページ目がない | ページごとにpage start／page endが対になっているか確認します。 |
| 画像が×になる | URLの到達性、data URI、CORS、サーバー側許可リストを確認します。 |
| 日本語が欠ける | 配置したフォント、フォントマップ、WASMとWeb資源の版を確認します。 |
| WASMが読み込めない | `.wasm`のURL、`application/wasm`、キャッシュ、同一リリースのJSを確認します。 |
| PDFだけ失敗する | バーコードの値、入力サイズ、画像サイズ、タイムアウト、サーバーログの要求IDを確認します。表せない値のバーコードがあると、PDFは対象のバーコードと値を示すエラーで止まります（プレビューでは枠に赤で「バーコードエラー」と表示します）。 |
| サーバーPDFが400・413・503になる | 応答JSONの`error`を見ます。400は入力の誤り、413は大きすぎる要求、503は混雑です。第8.2節の表を参照してください。 |

## 13. 配置・更新・ライセンス

ラッパー、プレビュアー、JavaScript／TypeScript、CSS、WASMは同じReports.web配布版から取り出してください。HTMLだけ、またはラッパーだけを別バージョンへ差し替えると、PREPEJや公開APIの互換性を判断できなくなります。

現行の製品版への切替は、購入者のメールアドレスを含む電子署名付きの `reports-web.license` を、実際に帳票を出力する側へ配置して行います。購入者ごとにWASMを差し替える手順ではありません。ブラウザーとサーバーでは配置先が異なるため、製品版を使い始めるに従ってください。ライセンスファイルを公開サンプルやソース管理へ混入させないでください。正式な動作環境、使用許諾、更新履歴、問い合わせ先は共通製品マニュアルを正本とします。開発ライセンスは開発用PC 1台につき1ライセンスです。